

MCP Agora

MCP Server with cross-agent persistent memory for AI agent fleets

Antonio Cioffi

2026

Abstract

MCP Agora is a portfolio/learning project implementing an MCP Server (Model Context Protocol) with cross-agent persistent memory for personal AI agent fleets. It uses ChromaDB for vector indexing, sentence-transformers for semantic embeddings (all-MiniLM-L6-v2 model, 384 dimensions), SQLite for metadata and persistent cache, and FastMCP to expose 8 MCP tools. The architecture is organized into 7 layers (Transport, Protocol, Router, Memory, Cache, Backend Connector, Embedding) with semantic routing to external MCP backends (GitHub, Playwright). The project has 61 automated tests across 10 files, zero mocks, using real ChromaDB and SQLite in isolated temp directories.

Repository: <https://github.com/cioffiAI/mcp-agora>

Contents

1	Introduction and Goals	2
1.1	Product Goals	2
1.2	Learning Goals	2
1.3	Positioning	2
2	Technology Stack	2
2.1	Technology Choices	3
3	Architecture	4
3.1	Architecture Diagram	4
3.2	Layer Description	6
4	API Tools	7
5	Data Schema	7
5.1	ChromaDB: Vector Store	7
5.2	SQLite: Relational Database	8
6	Testing	8
7	Design Decisions	9
8	Conclusions and Roadmap	10
8.1	Completed Roadmap	10
8.2	Future Roadmap	10
8.3	Success Metrics	10
A	Complete config.yaml	11
B	Project Structure	11
C	Commit History	12

1 Introduction and Goals

MCP Agora is a portfolio/learning project implementing an MCP Server (Model Context Protocol) with cross-agent persistent memory for personal AI agent fleets. It stems from a real need to solve a concrete problem: modern AI agents (Claude Code, Codex CLI, ChatGPT, Gemini CLI) operate in isolated sessions with no shared memory. Each agent repeats searches, re-contextualizes information, and duplicates work that another agent has already done.

1.1 Product Goals

- One agent saves knowledge → another retrieves it (zero duplication).
- A query already made → cached response (zero recomputation).
- A mistake already made → not repeated (decision memory).
- 4+ agents sharing context without individual configuration.

1.2 Learning Goals

- MCP Protocol deep dive: compliant MCP Server implementation using official SDK.
- Vector search & embeddings: ChromaDB + sentence-transformers integration.
- Layered system design: modular architecture with 7 separate layers and abstract interfaces.
- Async Python: asyncio for MCP transport concurrency.
- Advanced SQLite: relational schema for metadata, provenance, analytics.
- Caching strategies: in-memory TTLCache + persistent SQLite.

1.3 Positioning

Agora does not compete with enterprise projects like ContextForge (IBM), MetaMCP, AutoMem, or mcp-memory-service. It is a portfolio project demonstrating competence in AI infrastructure, MCP protocol, vector search, semantic routing, and system design, solving a real problem in personal agent workflows. Its value lies not in concept originality, but in execution quality: 2026 stack (MCP, vectors, embedding), clean architecture (7 layers, abstract dependencies, testable), and a real problem (isolated agent memory).

2 Technology Stack

The project is entirely developed in Python 3.13+ with uv as package manager. The following table summarizes the main stack components.

Table 1: Main technology stack components.

Component	Technology	Version
Runtime	Python	≥ 3.13
Package Manager	uv	≥ 0.11
MCP Framework	FastMCP (mcp SDK)	$\geq 1.0.0$
Vector Store	ChromaDB (PersistentClient)	latest
Embeddings	sentence-transformers	$\geq 3.0.0$
Embedding Model	all-MiniLM-L6-v2	384d
Cache L1	cachetools TTLCache	latest
Cache L2 / DB	SQLite3 (stdlib)	built-in
Config	PyYAML	latest
HTTP Client	httpx	latest
PDF Generation	fpdf2	2.8.7
Testing	pytest + pytest-asyncio	$\geq 9.0.3$

2.1 Technology Choices

FastMCP

The official Anthropic SDK offers FastMCP, a high-level wrapper that simplifies MCP server creation. Compared to low-level Server, it requires less boilerplate and allows focusing on application logic.

ChromaDB

Embedded vector database, zero config, zero cloud. Supports ANN index (HNSW) for $O(\log n)$ search and cosine distance. PersistentClient ensures disk persistence without external server.

sentence-transformers

all-MiniLM-L6-v2 model: 384 dimensions, about 80MB, sufficient for MVP. Lazy-loaded (not at import) and cached in `~/.cache/agora/models/`.

SQLite

For metadata, provenance and L2 cache. Zero configuration, file-based, included in Python standard library. Each operation opens and closes a connection (thread-safe).

3 Architecture

MCP Agora’s architecture is organized into 7 separate layers, each with well-defined responsibilities and abstract interfaces. This modular approach allows testing, replacing, and evolving each layer independently.

3.1 Architecture Diagram

MCP Agora - Local Shared-Memory Architecture

Python-only MCP server for cross-agent persistent memory, cache, routing, and external MCP connectors

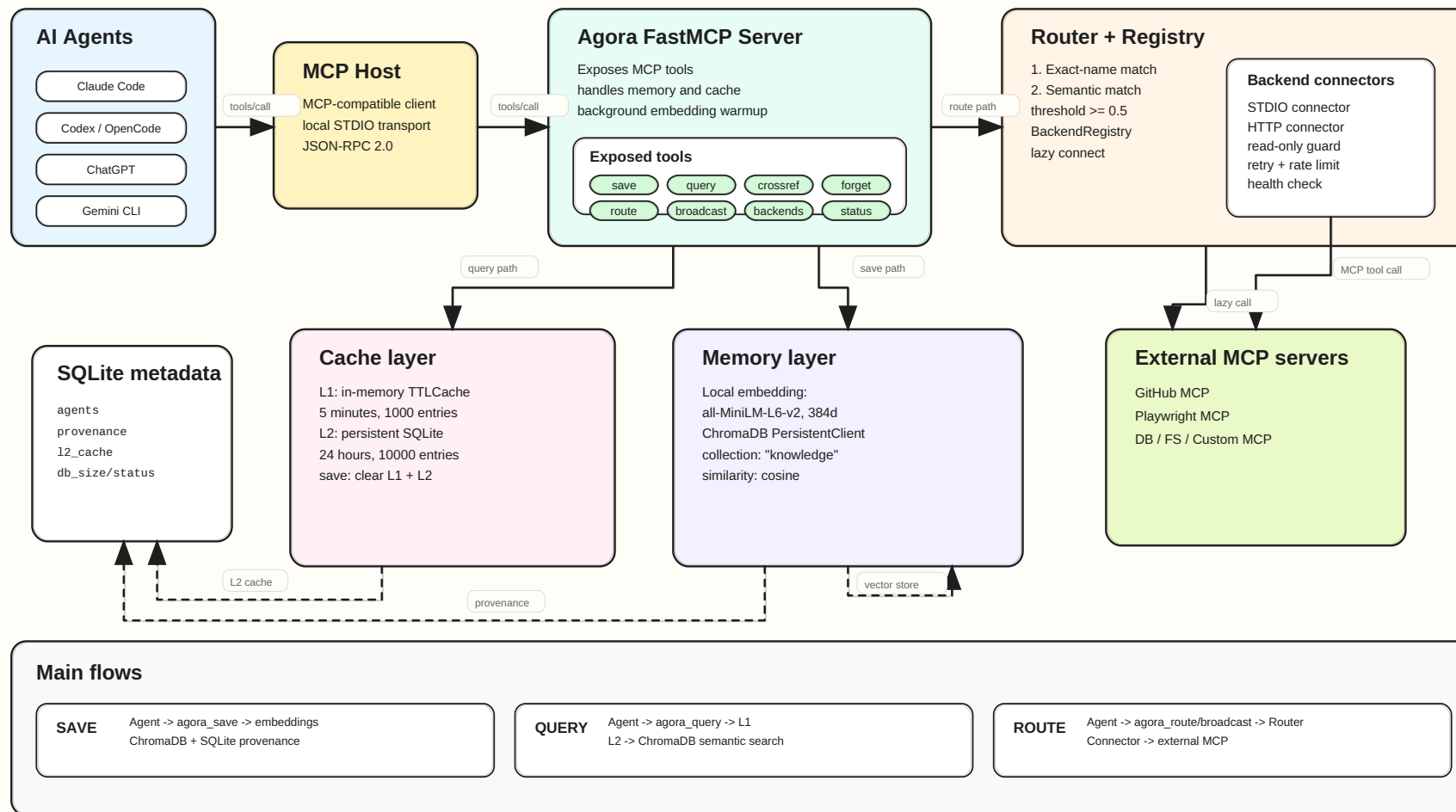


Figure 1: Local shared-memory architecture of MCP Agora.

3.2 Layer Description

Layer 1: Transport Layer

Manages communication with agents via JSON-RPC 2.0. Supports two transports: STDIO (local, stdin/stdout, default) for agents like Claude Code and Codex CLI; Streamable HTTP (remote, HTTP POST + SSE) for remote or multi-session agents.

Layer 2: Protocol Layer

Exposes standard MCP primitives: tools/list to discover available tools, tools/call to invoke them. Agora exposes 8 tools (see Section 4).

Layer 3: Router Layer

Decision-making core of the system. When an agent calls `agora_route`, the router first attempts an exact name match (case-insensitive), then a semantic match via cosine similarity on backend description embeddings (threshold ≥ 0.5). If no match, returns error. Broadcast routing (`agora_broadcast`) sends the request in parallel to all backends, limited to read-only tools only.

Layer 4: Memory Layer

Composed of two sub-layers: Vector Index (ChromaDB, knowledge collection, cosine space, 384 dimensions) for semantic search, and Relational DB (SQLite3, tables `agents`, `provenance`, `l2_cache`) for structured metadata. Knowledge retention is permanent (until deleted with `agora_forget`). L2 cache has configurable TTL (default 24h).

Layer 5: Cache Layer

Two cascading cache levels. L1: `cachetools TTLCache` in-memory (1000 entries, 5 minutes TTL). L2: SQLite-backed on disk (10000 entries, 24h TTL). Cache key = SHA256 of full JSON-serialized request with `sort_keys=True`. On `agora_save` and `agora_forget`, both caches are invalidated (full clear). Cascade: L1 → L2 → ChromaDB. On ChromaDB miss, both caches are populated.

Layer 6: Backend Connector Layer

Each external MCP backend is represented by a connector implementing the abstract `BackendConnector` interface. Two implementations: `StdioConnector` (sub-process MCP via `stdio_client` + `ClientSession`, lazy connect, `asyncio.timeout`) and `HttpConnector` (streamable HTTP MCP via `streamablehttp_client`). Connection is lazy: the connector connects on first use, not at startup. Read-only enforcement: if a backend is marked `read_only`, only tools with whitelisted prefixes (`list_`, `get_`, `fetch_`, `read_`, `search_`, `find_`, `query_`, `describe_`, `show_`) can be called. Mutative tools raise `ReadOnlyBlockedError`.

Layer 7: Embedding Layer

Abstract `EmbeddingProvider` with concrete `SentenceTransformerProvider` implementation using `all-MiniLM-L6-v2` (384 dimensions). Lazy loading: model loads on first use, not on module import. Cache in `~/.cache/agora/models/`. First call about 17s (model download + torch). Subsequent calls <1s. The `warmup()` method pre-loads the model at server startup.

4 API Tools

Agora exposes 8 MCP tools, registered via `@mcp.tool()` decorator on `FastMCP`. All tools accept and return JSON-serializable dictionaries.

Table 2: MCP tools exposed by Agora.

Tool	Parameters	Behavior	Cache
<code>agora_save</code>	<code>content</code> , <code>tags</code> , <code>agent</code> , <code>session</code> , <code>confidence</code>	Saves knowledge in ChromaDB + provenance in SQLite. Registers agent.	Clear L1+L2
<code>agora_query</code>	<code>query</code> , <code>top_k</code> (=5)	Semantic search. Cascade: L1 → L2 → ChromaDB.	Populates L1+L2 on miss
<code>agora_route</code>	<code>target</code> , <code>tool</code> , <code>arguments</code>	Routing to MCP backend. Exact match → semantic (≥ 0.5).	No
<code>agora_broadcast</code>	<code>tool</code> , <code>arguments</code>	Parallel broadcast (asyncio.gather) to all backends. Read-only only.	No
<code>agora_backends</code>	(none)	Lists configured backends + connection status.	No
<code>agora_crossref</code>	<code>query</code> , <code>entry_id</code> , <code>top_k</code>	Cross-agent correlations. By query: groups by agent. By ID: finds correlated entries.	No
<code>agora_forget</code>	<code>entry_ids</code> , <code>tags</code> , <code>agent</code> , <code>dry_run</code>	Deletes memory. Composite filters. <code>dry_run</code> preview. Clear L1+L2.	Clear L1+L2
<code>agora_status</code>	(none)	Statistics: <code>entries</code> , <code>agents</code> , L1+L2 cache, backend, db_size.	No

5 Data Schema

5.1 ChromaDB: Vector Store

Single knowledge collection with cosine metric space. Documents are saved texts, embeddings are 384-dimensional vectors from `all-MiniLM-L6-v2`, and metadata includes tags (CSV format), `created_at` (ISO 8601), and `agent`.

The VectorStore wrapper (agora/memory/vector_store.py) abstracts the ChromaDB API and handles collection creation and serialization automatically.

5.2 SQLite: Relational Database

Three tables for structured metadata: agents (agent registry), provenance (knowledge entry traceability), l2_cache (persistent disk cache).

Listing 1: SQLite relational schema.

```
CREATE TABLE IF NOT EXISTS agents (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    agent_type TEXT DEFAULT 'unknown',
    first_seen TEXT NOT NULL,
    last_seen TEXT NOT NULL,
    sessions_count INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS provenance (
    entry_id TEXT NOT NULL,
    source_agent TEXT NOT NULL,
    source_session TEXT NOT NULL,
    confidence REAL DEFAULT 0.5,
    created_at TEXT NOT NULL,
    PRIMARY KEY (entry_id, source_agent, source_session)
);

CREATE TABLE IF NOT EXISTS l2_cache (
    cache_key TEXT PRIMARY KEY,
    result_json TEXT NOT NULL,
    ttl_seconds INTEGER NOT NULL DEFAULT 86400,
    hit_count INTEGER DEFAULT 0,
    created_at TEXT NOT NULL,
    expires_at TEXT NOT NULL
);
```

6 Testing

61 tests across 10 files, zero mocks – real ChromaDB and SQLite in isolated temp directories per test (pytest function-scoped fixtures with temp directory). Timing not used as metric (flaky on Windows); `cache.stats()['hit_count']` is used instead.

Table 3: Automated test coverage.

File	No.	Scope
test_embedding.py	3	Unit: dimension 384, vector format, retrieval order
test_memory.py	5	Unit: ChromaDB add/query/delete/multi/integration
test_cache.py	5	Unit: L1 set/get/expiry/hit_count/clear

File	No.	Scope
test_l2_cache.py	7	Unit: L2 set/get/miss/expiry/hit_count/stats/clear/prune
test_provenance.py	7	Unit: agent registry + provenance lifecycle
test_protocol.py	6	Integration: FastMCP wiring, save→query, cache hit, L2 fallback
test_routing.py	9	Unit: cosine similarity, exact/semantic/no match, warmup
test_connectors.py	8	Unit: STDIO/HTTP properties, timeout, health, read-only
test_graceful.py	6	Graceful: backend failure, health cascade, retry, rate limit
test_mcp_smoke.py	5	Smoke: MCP ClientSession, full stack

7 Design Decisions

The following design decisions were made consciously, with explicit rationale. They should not be changed without discussion.

Table 4: Design decisions and rationale.

Decision	Rationale
FastMCP (not low-level Server)	Faster development, cleaner code
ChromaDB (not SQLite-VSS)	More mature, better documentation
Cache clear-all on save	Simple, safe, sufficient for MVP
No semantic cache	Two similar queries can be operationally different
all-MiniLM-L6-v2	384d, lightweight, sufficient for MVP
Lazy backend connect	No startup cost for idle backends
Routing: exact → semantic (≥ 0.5)	Exact match tried first, semantic fallback
Read-only prefix heuristic	list_, get_, fetch_, read_, search_, find_...
SQLite connection per-call	Thread-safe, not optimized for high concurrency
Manual cosine similarity (no numpy)	Zero external dependencies for routing

8 Conclusions and Roadmap

8.1 Completed Roadmap

Phase 1 - Core Memory

save/query/status with ChromaDB, sentence-transformers, TTLCache L1, FastMCP.

Phase 2 - Routing + Connectors

Semantic + exact Router, StdioConnector, HttpConnector, BackendRegistry, broadcast, read-only enforcement.

Phase 3 - Cross-Agent Memory

Provenance tracking (SQLite), persistent L2 cache, crossref, forget, agent/session/-confidence params on save.

Phase 4 - Robustness

Structured logging, backend health checks (healthy/unhealthy/dead), configurable retry, rate limiting, non-blocking startup, WarmingUpError grace.

8.2 Future Roadmap

- Phase 5: Full README, real examples, GitHub + PyPI publication (in progress).
- Chunking: multi-paragraph document handling with semantic boundary split.

8.3 Success Metrics

Table 5: Success metrics.

Metric	Target
Cache hit rate	> 40% (MVP) / > 60% (target)
Average query latency	< 1000ms
Cache hit latency	< 50ms
Semantic routing coverage	>7%
Cross-agent knowledge reuse	20
61 tests, 0 mocks	No, it is a fact.

MCP Agora is not an invention. It is an engineering exercise on a real problem (isolated agent memory) with modern tools (MCP, vector search, embeddings). If it solves the problem of 4 agents not sharing memory, it has already won.

A Complete config.yaml

The config.yaml file in the project root directory controls server, storage, cache, embedding, and backend configuration.

Listing 2: Complete config.yaml file.

```
agora:
  name: "Agora"
  version: "0.4.0"

storage:
  chroma_path: "~/.agora/chroma"
  db_path: "~/.agora/agora.db"

cache:
  l1_max_entries: 1000
  l1_ttl_seconds: 300
  l2_max_entries: 10000
  l2_ttl_seconds: 86400

embedding:
  provider: "sentence-transformers"
  model: "all-MiniLM-L6-v2"

backends:
  - name: "github"
    transport: "stdio"
    command: ["npx", "-y", "@modelcontextprotocol/server-github"]
    description: "GitHub API: issues, PRs, repos, code search"
    read_only: false
    timeout_seconds: 15
    max_retries: 3
    retry_delay: 1.0
    rate_limit_rps: 5
    env:
      GITHUB_TOKEN: "${GITHUB_TOKEN}"
  - name: "playwright"
    transport: "stdio"
    command: ["npx", "@playwright/mcp@latest"]
    description: "Browser automation: navigate, click, screenshot, forms"
    read_only: true
    timeout_seconds: 30
    max_retries: 2
    retry_delay: 0.5
    rate_limit_rps: 2
```

B Project Structure

Listing 3: Project structure.

```
mcp-agora/
|-- pyproject.toml      # Dependencies, build, entry point "agora"
|-- config.yaml        # Server configuration + backend
|-- agora/
```

```

|-- main.py          # Entry point
|-- server.py        # FastMCP + 8 tools
|-- config.py        # YAML loader
|-- registry.py      # BackendRegistry
|-- connectors/
|   |-- base.py      # BackendConnector ABC
|   |-- stdio.py     # StdioConnector
|   +-- http.py      # HttpConnector
|-- routing/
|   +-- router.py    # Semantic + exact router
|-- embedding/
|   |-- base.py      # EmbeddingProvider ABC
|   +-- sentence.py  # sentence-transformers
|-- memory/
|   +-- vector_store.py # ChromaDB wrapper
|-- cache/
|   |-- l1_memory.py # TLCache
|   +-- l2_cache.py  # SQLite cache
+-- db/
|   +-- database.py  # SQLite schema + ops
-- tests/
|   |-- test_embedding.py # 3 tests
|   |-- test_memory.py   # 5 tests
|   |-- test_cache.py    # 5 tests
|   |-- test_l2_cache.py # 7 tests
|   |-- test_provenance.py # 7 tests
|   |-- test_protocol.py # 6 tests
|   |-- test_routing.py  # 8 tests
|   |-- test_connectors.py # 9 tests
|   |-- test_mcp_smoke.py # 8 tests
|   +-- _echo_server.py
-- docs/
|   +-- generate_pdf.py
-- README.md
+-- LICENSE (MIT)

```

C Commit History

Commit history on the master branch of the GitHub repository.

Listing 4: Commit history.

```

14dbdc5 Initial commit: MCP Agora Phase 1 complete
e209f40 Add MIT License
005590b Add agora usage instructions to AGENTS.md
5a64014 Phase 2: Routing + Backend Connectors
12e54ae docs: update README and AGENTS.md for Phase 2
0c36648 feat: HTTP connector, read_only, broadcast, timeout
b03abbe fix: ReadOnlyBlockedError public, parallel broadcast
dab75a2 feat: Phase 3 cross-agent memory
7f01873 docs: update ARCHITECTURE.md with Phase 2/3 details, add ruff linter
, generate PDF
c4aa01c feat: Phase 4 robustness - logging, health check, retry, rate limit,
graceful tests
40e86f6 fix: non-blocking startup + WarmingUpError grace
90765e8 chore: add *.log and chroma.sqlite3 to .gitignore

```

